



دانشگاه صنعتی همدان

دانشکده فنی و مهندسی

گروه کامپیوتر

پروژه پیاده سازی شبیه سازی جام جهانی

دانشجو:

محمد مهدی رحیمی طبالوندانی

استاد راهنما:

فاطمه امیری

مرداد ۱۴۰۵

مقدمه

برای توضیح این پروژه می‌شد از اولین خط برنامه شروع کرد و تا منوی خروج جلو رفت، اما این مدل گزارش خیلی زود به بازگویی کد تبدیل می‌شد. من مسیر دیگری را انتخاب کردم: هر جا برنامه باید یک تصمیم مهم بگیرد، همان تصمیم را به شکل یک پرسش مطرح کرده‌ام و بعد نشان داده‌ام کدام قسمت کد جواب آن پرسش است.

گزارش در سه قسمت تنظیم شده است. قسمت اول درباره شکل‌گیری داده و تقسیم مسئولیت بین فایل‌هاست. قسمت دوم قواعدی را دنبال می‌کند که یک فهرست ۳۲ تیمی را به قهرمان می‌رسانند. قسمت سوم هم سراغ منو، محاسبه درصد قهرمانی و آزمونی می‌رود که باگ برکت را کنترل می‌کند. این تقسیم‌بندی عمداً با ترتیب اجرای منو یکی نیست.

عدد گل و نام قهرمان در اجرای واقعی می‌تواند عوض شود، چون بخشی از برنامه تصادفی است. در نتیجه، خروجی‌های نوشته‌شده در این گزارش نمونه‌اند؛ چیزی که باید ثابت بماند قانون بازی، ترتیب برکت و جدابودن گزارش آماری از جام ذخیره‌شده است.

قسمت اول: تصمیم‌های پایه در طراحی برنامه

۱. نقشه ارتباط فایل‌ها

پرسش این بخش: چرا برنامه به چند فایل تقسیم شده و نقطه شروع آن کجاست؟

فایل منو فقط با WorldCupSimulator صحبت می‌کند و از جزئیات ساخت مسابقه یا گروه خبر ندارد. در لایه بعد، کلاس اصلی چهار کلاس Team، Match، Group و KnockoutStage را کنار هم قرار می‌دهد. این ارتباط یک‌طرفه باعث شد هنگام پیدا کردن خطا بدانیم باید سراغ کدام فایل برویم و بی‌دلیل همه برنامه را تغییر ندهیم.

نمونه‌ای از پیاده‌سازی

```
from ClassWorldCupSimulator import WorldCupSimulator

from ClassGroup import Group
from ClassKnockoutStage import KnockoutStage
from ClassMatch import Match
from ClassTeam import Team
```

آنچه در اجرای نمونه بررسی شد

```
Main module loaded.
WorldCupSimulator dependencies: 4 classes
```

۲. قرارداد داده ورودی

پرسش این بخش: یک سطر از فایل تیم‌ها چه زمانی اجازه ورود به برنامه را دارد؟

خواندن فایل به معنی اعتماد کردن به آن نیست. هر سطر باید نام غیر خالی و سه مقدار عددی داشته باشد؛ قدرت حمله و دفاع در بازه ۱ تا ۱۰۰ و رتبه در بازه ۱ تا ۳۲ قرار می‌گیرد. نام و رتبه نیز نباید تکراری باشند. تا وقتی تمام سطرها این قرارداد را رعایت نکرده‌اند، فهرست اصلی تیم‌ها دست‌نخورده باقی می‌ماند.

```
# Har satr pish az sakhte-shodan-e shey-e Team etebarsanji mishavad.
for row in reader:
    if (
        row["name"] is None
        or row["attack"] is None
        or row["defense"] is None
        or row["rank"] is None
    ):
        print("Khata: Yeki az satr-haye file naghesh ast.")
        return False

    name = row["name"].strip()
    attack_text = row["attack"].strip()
    defense_text = row["defense"].strip()
    rank_text = row["rank"].strip()

    if name == "":
        print("Khata: Nam-e yek team khali ast.")
        return False

    if not (
        attack_text.isdigit()
        and defense_text.isdigit()
        and rank_text.isdigit()
    ):
        print(
            "Khata: Attack, defense va rank "
            "bayad adad-e sahih bashand."
        )
        return False

    if not (1 <= attack <= 100 and 1 <= defense <= 100):
        print(
            "Khata: Attack va defense bayad "
            "bein-e 1 ta 100 bashand."
        )
        return False

    if not 1 <= rank <= 32:
        print("Khata: Rank-e team bayad bein-e 1 ta 32 bashad.")
        return False

    if name in team_names or rank in team_ranks:
        print("Khata: Nam-e team ya rank dar file tekrari ast.")
        return False

    team_names.add(name)
    team_ranks.add(rank)
    loaded_teams.append(Team(name, attack, defense, rank))
```

آنچه در اجرای نمونه بررسی شد

Empty name -> rejected
 Attack = 105 -> rejected
 Duplicate rank -> rejected

۳. تبدیل ویژگی تیم به شانس گل و پناستی

پرسش این بخش: قدرت‌های ثبت‌شده در فایل چطور وارد نتیجه مسابقه می‌شوند؟

برای گل، سهم حمله تیم با ضعف دفاع حریف جمع می‌شود و مقدار λ را می‌سازد. برای پناستی هم اختلاف حمله و دفاع در فرمول احتمال اثر می‌گذارد. محدود کردن احتمال بین ۰.۶ و ۰.۹ نکته کوچکی است، ولی جلوی عددهای نامعتبر و برتری صددرصدی یک تیم را می‌گیرد.

```
def _calculate_goal_lambda(self, opponent):
    # Ghodrat-e hamle-ye team va defa-e harif lambda ra misazand.
    attack_part = (self.attack / 100) * 1.5
    defense_part = (1 - opponent.defense / 100) * 0.8
    return attack_part + defense_part

def _penalty_probability(self, opponent):
    # Ehtemal-e penalty tebghe formule file-e proje mohasebe mishavad.
    probability = 0.75 + (self.attack - opponent.defense) / 250

    # Ehtemal bayad hatman dar bazeye 0.6 ta 0.9 bemanad.
    if probability < 0.6:
        probability = 0.6
    elif probability > 0.9:
        probability = 0.9

    return probability
```

آنچه در اجرای نمونه بررسی شد

Iran lambda against Japan: 1.39
Iran penalty probability: 0.76

۴. ساخت یک نتیجه قابل خواندن

پرسش این بخش: چطور گل، پنالتی و نام برنده در یک خروجی روشن جمع می‌شوند؟

کلاس Match نتیجه را یک بار می‌سازد تا منو و همه دورهای حذفی همان متن را مصرف کنند. ابتدا گل‌های زمان بازی نوشته می‌شود؛ اگر مسابقه به پنالتی رسیده باشد، نتیجه آن داخل پرانتز می‌آید و در پایان نام برنده اضافه می‌شود. به این ترتیب گل پنالتی با گل زده و خوردۀ جدول مخلوط نمی‌شود.

نمونه‌ای از پیاده‌سازی

```
def result_text(self):
    """Natije-ye bazi ra be matn-e ghabel-e namayesh tabdil mikonad.
    Voroodiha: Nadarad.
    Khorooji: Natije, penalty-e ehtemali va barande (str).
    """
    # Natije-ye gol-haye asli dar aval-e matn gharar migirad.
    text = (
        self.team1.name + " " + str(self.goals1) + " - "
        + str(self.goals2) + " " + self.team2.name
    )

    # Agar bazi be penalty reside bashad, natije-ye penalty ezafe mishavad.
    if self.penalties1 is not None:
        text += (
            " (Penalty: " + str(self.penalties1) + " - "
            + str(self.penalties2) + ")"
        )

    if self.winner is not None:
        text += " -> Barande: " + self.winner.name

    return text
```

آنچه در اجرای نمونه بررسی شد

Japan 1 - 1 Mexico (Penalty: 3 - 4) -> Barande: Mexico

قسمت دوم: از جدول گروه تا قهرمان جام

۵. رتبه‌بندی‌ای که دوباره قرعه نمی‌کشد

پرسش این بخش: اگر دو تیم در همه معیارها برابر باشند، جدول چگونه ثابت می‌ماند؟

معیارهای اصلی جدول به ترتیب امتیاز، تفاضل گل و گل زده هستند. پیش از مرتب‌سازی، یک ترتیب تصادفی برای حالت تساوی کامل ساخته می‌شود؛ اما نتیجه در ویژگی `ranking` ذخیره می‌ماند. بنابراین چاپ دوباره جدول یا انتخاب دو تیم صعودکننده قرعه تازه‌ای انجام نمی‌دهد.

نمونه‌ای از پیاده‌سازی

```
def _calculate_ranking(self):
    # random.sample tartib-e avalie ra baraye ghore-ye tasavi moshakhas mikonad.
    return sorted(
        random.sample(self.teams, len(self.teams)),
        key=lambda team: (
            team.points,
            team.goal_difference(),
            team.goals_for
        ),
        reverse=True
    )

def get_ranking(self):
    """Jadval-e grooh ra tebghe chahar meyar barmigardanad.
    Voroodiha: Nadarad.
    Khorooji: Team-ha az rotbe-ye aval ta chaharom (list).
    """
    # Ranking-e zakhire-shode az ghore-keshi-e dobare jologiri mikonad.
    if len(self.ranking) == 0:
        self.ranking = self._calculate_ranking()
    return self.ranking

def advance_teams(self):
    """Do team-e aval-e grooh ra entekhab mikonad.
    Voroodiha: Nadarad.
    Khorooji: Team-e aval va dovom (tuple).
    """
    ranking = self.get_ranking()
    return ranking[0], ranking[1]
```

آنچه در اجرای نمونه بررسی شد

```
1. Spain - 7 points
2. Senegal - 5 points
Advanced: Spain, Senegal
```

۶. سهم هر سید در قرعه‌کشی

پرسش این بخش: چطور مطمئن می‌شویم هر گروه دقیقاً یک تیم از هر سید دارد؟

تیم‌ها با رتبه مرتب و به چهار فهرست هشت‌تیمی تقسیم می‌شوند. هر فهرست جداگانه به هم می‌ریزد و عضو هم‌شماره چهار سید وارد یک گروه می‌شود. در نتیجه گروه A تا H همگی چهار تیم دارند، هیچ تیمی تکرار نمی‌شود و تعادل اولیه سیدها حفظ می‌شود.

```
# Team-ha bar asas rank be chahar pot-e hasht-team-i taghsim mishavand.
sorted_teams = sorted(self.teams, key=lambda team: team.rank)
pots = [
    sorted_teams[0:8],
    sorted_teams[8:16],
    sorted_teams[16:24],
    sorted_teams[24:32]
]
shuffled_pots = []

# Tartib-e har pot bedoon-e tkrar tasadofi mishavad.
for pot in pots:
    shuffled_pots.append(random.sample(pot, len(pot)))

group_names = ["A", "B", "C", "D", "E", "F", "G", "H"]
self.groups = []
self._clear_tournament_results()

# Har grooh daghighan yek team az har pot migirad.
for group_index in range(8):
    group_teams = []
    for pot_index in range(4):
        team = shuffled_pots[pot_index][group_index]
        team.group = group_names[group_index]
        group_teams.append(team)
    self.groups.append(Group(group_names[group_index], group_teams))
```

آنچه در اجرای نمونه بررسی شد

Group A pot numbers: [1, 2, 3, 4]

Group H pot numbers: [1, 2, 3, 4]

۷. مرز میان اجرای گروهی و جام تازه

پرسش این بخش: چرا اجرای دوباره مرحله گروهی نباید نتیجه جام قبلی را نگه دارد؟

وقتی کاربر مرحله گروهی را از نو اجرا می‌کند، امتیازها و نتایج دور قبلی نباید با اجرای تازه جمع شوند. متد `run_group_stage` اول وجود هشت گروه را کنترل می‌کند، سپس آمار و حافظه دوره‌های حذفی را پاک می‌کند و بعد بازی‌ها را جلو می‌برد. این ترتیب، مرز یک اجرای تازه را مشخص می‌کند.

نمونه‌ای از پیاده‌سازی

```
def _play_group_stage(self, display_tables):
    # Hame-ye grooh-ha bazi mikonand va dar sorat-e niaz chap mishavand.
    for group in self.groups:
        group.play_all_matches()
        if display_tables:
            group.display_table()
    self.group_stage_completed = True

def run_group_stage(self):
    """Marhale-ye groohi va jadval-e har hasht grooh ra ejra mikonad.
    Voroodiha: Nadarad.
    Khorooji: Movafaghiat-e ejra (bool).
    """
    if len(self.groups) != 8:
        print("Ebtada ghorekeshi-e grooh-ha ra anjam dahid.")
        return False

    # Ejra-ye jadid nabayad amar ya bracket-e ghabli ra negah darad.
    self._reset_all_stats()
    self._clear_tournament_results()
    self._play_group_stage(True)
    return True
```

آنچه در اجرای نمونه بررسی شد

Before rerun: old champion exists

After rerun: group stage completed, old bracket cleared

۸. کنترل زوج‌های ثابت یک‌هشتم

پرسش این بخش: از کجا معلوم است مسیر براکت مطابق A_1-B_2 و B_1-A_2 ساخته شده؟

برای این قسمت فقط دیدن نام هشت مسابقه کافی نیست؛ باید خود اشیای **Team** در جای درست قرار گرفته باشند. آزمون زوج اول و پنجم براکت را با تیم‌های صعود کرده مقایسه می‌کند. اگر حتی یکی از اندیس‌های ساخت براکت جابه‌جا شود، این کنترل شکست می‌خورد و خطا پیش از ادامه جام دیده می‌شود.

نمونه‌ای از پیاده‌سازی

```
def test_knockout_starts_with_required_fixed_pairs(self):
    """Pair-haye A1-B2 va B1-A2 ra dar bracket barrasi mikonad."""
    self.simulator.seed_and_draw_groups(False)
    self.simulator._reset_all_stats()
    self.simulator._play_group_stage(False)
    self.simulator.setup_knockout_bracket()

    advanced = []
    for group in self.simulator.groups:
        advanced.append(group.advance_teams())

    first_match = self.simulator.round_of_16.matches[0]
    fifth_match = self.simulator.round_of_16.matches[4]
    self.assertIs(first_match.team1, advanced[0][0])
    self.assertIs(first_match.team2, advanced[1][1])
    self.assertIs(fifth_match.team1, advanced[1][0])
    self.assertIs(fifth_match.team2, advanced[0][1])
```

آنچه در اجرای نمونه بررسی شد

Match 1 is A1-B2: True
Match 5 is B1-A2: True

۹. پایان‌دادن به تساوی حذفی

پرسش این بخش: اگر پنج پنالتی اول مساوی شود، برنامه چگونه به برنده می‌رسد؟

هر تیم پنج ضربه اولیه می‌زند. اگر تعداد گل‌ها برابر باشد، حلقه پنالتی ناگهانی شروع می‌شود و در هر دور هر دو تیم یک ضربه می‌زنند. حلقه فقط وقتی تمام می‌شود که تساوی شکسته شده باشد؛ پس مرحله حذفی برخلاف گروهی همیشه یک تیم برنده برمی‌گردد.

نمونه‌ای از پیاده‌سازی

```
def _simulate_penalty_shootout(self, opponent):
    self.probability = self._penalty_probability(opponent)
    opponent_probability = opponent._penalty_probability(self)
    self.penalties = 0
    opponent_penalties = 0

    # Har team panj zarbeye avalie ra mizand.
    for shot_number in range(5):
        if random.random() < self.probability:
            self.penalties += 1
        if random.random() < opponent_probability:
            opponent_penalties += 1

    # Dar har dor-e nagahani, har do team yek zarbeye kamel mizanand.
    while self.penalties == opponent_penalties:
        if random.random() < self.probability:
            self.penalties += 1
        if random.random() < opponent_probability:
            opponent_penalties += 1

    return self.penalties, opponent_penalties
```

آنچه در اجرای نمونه بررسی شد

Five kicks: 4 - 4
Sudden death: 5 - 4
Winner determined

۱۰. تبدیل فهرست برندگان به قهرمان

پرسش این بخش: چطور تعداد تیم‌ها در هر دور نصف می‌شود و ترتیب مسیر حفظ می‌ماند؟

برندگان به همان ترتیبی که از دور قبلی برگشته‌اند، دوباره دو جفت می‌شوند. تابع `build_next_round` این کار را برای یک‌چهارم، نیمه‌نهایی و فینال تکرار می‌کند. در پایان فهرست برندگان فینال فقط یک عضو دارد و همان شیء در `champion` ذخیره می‌شود.

نمونه‌ای از پیاده‌سازی

```
def _build_next_round(self, round_name, winners):
    matches = []

    # Har do barande-ye motavali yek bazi-e dor-e badi ra misazand.
    for index in range(0, len(winners), 2):
        matches.append(Match(winners[index], winners[index + 1], True))

    return KnockoutStage(round_name, matches)

def _play_stage(self, stage, display_results):
    # Marhale ejra mishavad va barandegan baraye dor-e badi bargasht dade mishavand.
    stage.play_round()
    if display_results:
        stage.display_results()
    return stage.get_winners()

def run_knockout_stage(self, display_results=True):
    """Tamam-e marhale-haye hazfi ra ta taein-e ghahreman ejra mikonad.
    Voroodiha: display_results (bool), ejaze-ye chap-e natayej.
    Khorooji: Team-e ghahreman ya None.
    """
    if self.round_of_16 is None:
        print("Ebteda bracket-e marhale-ye hazfi ra besazid.")
        return None

    # Barandegan-e har marhale pair-haye marhale-ye badi ra misazand.
    winners = self._play_stage(self.round_of_16, display_results)
    self.quarterfinals = self._build_next_round(
        "Yek-chaharom nahaei", winners
    )
    winners = self._play_stage(self.quarterfinals, display_results)
    self.semifinals = self._build_next_round("Nime-nahaei", winners)
    winners = self._play_stage(self.semifinals, display_results)
    self.final = self._build_next_round("Final", winners)
    self.champion = self._play_stage(self.final, display_results)[0]
    return self.champion
```

آنچه در اجرای نمونه بررسی شد

8 winners -> 4 matches
4 winners -> 2 matches
2 winners -> 1 final
Champion: Uruguay

۱۱. منویی که ترتیب کار را روشن می کند

پرسش این بخش: کاربر بدون دانستن نام متدها چگونه همه قابلیت ها را اجرا می کند؟

منو هفت انتخاب دارد و ترتیب آن از بارگذاری داده تا خروج طراحی شده است. نام هر گزینه به کاربر می گوید قرار است وضعیت جام عوض شود یا فقط یک گزارش ببیند. گزینه ۴ جام کامل می سازد، گزینه ۵ گزارش چند اجرای مستقل را نشان می دهد و گزینه ۶ براکت ذخیره شده را دوباره چاپ می کند.

نمونه ای از پیاده سازی

```
def display_menu():
    """Gozine-haye menu-ye asli ra chap mikonad.
    Voroodiha: Nadarad.
    Khorooji: Nadarad.
    """
    print("\n===== Shabih-saz-e Jam-e Jahani =====")
    print("1) Bargozari-e team-ha az file-e CSV")
    print("2) Anjam-e ghorekeshi-e grooh-ha")
    print("3) Ejra-ye marhale-ye groohi va namayesh-e jadval-ha")
    print("4) Ejra-ye kamel-e jam va namayesh-e ghahreman")
    print("5) Shabih-sazi-e chandbare va gozaresh-e darsad-e ghahremani")
    print("6) Namayesh-e bracket-e hazfi-e akharin jam-e kamel")
    print("7) Khorooj")

    # Gozine-haye 2 ta 6 bedoon-e team ejra nemishavand.
    elif choice in ["2", "3", "4", "5", "6"] and len(
        simulator.teams
    ) == 0:
        print("Ebtada team-ha ra bargozari konid.")
    elif choice == "2":
        simulator.seed_and_draw_groups()
    elif choice == "3":
        simulator.run_group_stage()
    elif choice == "4":
        simulator.run_full_simulation()
```

آنچه در اجرای نمونه بررسی شد

- 4) Full tournament
- 5) Championship percentages
- 6) Saved knockout bracket

۱۲. معنای درست درصد قهرمانی

پرسش این بخش: آیا تیم اول جدول درصد باید قهرمان براکت آخر باشد؟

خیر. درصد قهرمانی از تکرار تعداد زیادی جام کامل به دست می آید و احتمال تقریبی هر تیم را نشان می دهد. براکت فقط روایت یک اجرای مشخص است. برای جلوگیری از بازنویسی آن اجرا، تیم های آماری در یک WorldCupSimulator جدا ساخته می شوند و قهرمانی های آن شیء مستقل شمارش می شوند.

```

championship_counts = {}
statistics_simulator = WorldCupSimulator()

# Team-haye amari joda hastand ta bracket-e jam-e asli baznevisi nashavad.
for team in self.teams:
    championship_counts[team.name] = 0
    statistics_simulator.teams.append(
        Team(team.name, team.attack, team.defense, team.rank)
    )

# Har ejra yek jam-e kamel-e makhfi dar shabih-saz-e amari ast.
for simulation_number in range(num_simulations):
    champion = statistics_simulator.run_full_simulation(False)
    championship_counts[champion.name] += 1

percentages = {}
for team_name in championship_counts:
    count = championship_counts[team_name]
    percentages[team_name] = count * 100 / num_simulations

```

آنچه در اجرای نمونه بررسی شد

Argentina: 9.2%
 France: 8.8%
 Uruguay: 4.1%
 Sum: 100.0%

برداشت درست: بیشترین درصد، پیش‌بینی آماری است و جای نتیجهٔ فینال ذخیره‌شده را نمی‌گیرد. متفاوت‌بودن این دو نام نشانهٔ باگ نیست؛ عوض‌شدن براکت پس از گزینهٔ ۵ باگ است.

۱۳. آزمون بازگشت باگ براکت

پرسش این بخش: چطور ثابت می‌کنیم درصدگیری نتیجهٔ گزینهٔ ۴ را تغییر نداده است؟

آزمون ابتدا یک جام کامل اجرا می‌کند و قهرمان، چهار شیء مرحله و متن همهٔ مسابقه‌ها را کنار می‌گذارد. سپس هزار شبیه‌سازی آماری انجام می‌دهد و همان اطلاعات را دوباره می‌خواند. برابر بودن اشیا و متن‌ها نشان می‌دهد نه‌فقط نام قهرمان، بلکه کل مسیر براکت بدون تغییر مانده است.

```
def test_statistics_do_not_replace_saved_bracket(self):
    """Bug-e baznevisi-e bracket tavasot gozine-ye 5 ra barrasi mikonad."""
    champion = self.simulator.run_full_simulation(False)
    stages_before = [
        self.simulator.round_of_16,
        self.simulator.quarterfinals,
        self.simulator.semifinals,
        self.simulator.final,
    ]
    results_before = []

    for stage in stages_before:
        for match in stage.matches:
            results_before.append(match.result_text())

    with contextlib.redirect_stdout(io.StringIO()):
        percentages = self.simulator.most_likely_champion(1000)

    stages_after = [
        self.simulator.round_of_16,
        self.simulator.quarterfinals,
        self.simulator.semifinals,
        self.simulator.final,
    ]
    results_after = []

    for stage in stages_after:
        for match in stage.matches:
            results_after.append(match.result_text())

    self.assertIs(self.simulator.champion, champion)
    self.assertEqual(stages_after, stages_before)
    self.assertEqual(results_after, results_before)
    self.assertAlmostEqual(sum(percentages.values()), 100.0)
```

آنچه در اجرای نمونه بررسی شد

```
Champion unchanged: True
Stages unchanged: True
Results unchanged: True
Percentage sum: 100.0%
```

نتیجه‌گیری

چیزی که در پایان این پروژه برای من روشن شد این بود که تصادفی بودن با بی‌قاعده بودن فرق دارد. ممکن است هر بار تیم دیگری قهرمان شود، اما مسیر رسیدن به آن نتیجه باید همیشه قابل ردیابی باشد: داده کنترل شود، جدول یک بار رتبه‌بندی شود، زوج‌های برکت در جای خود بمانند و تساوی حذفی حتماً برنده پیدا کند.

اصلاح برکت هم در اصل یک مسئله ساده مالکیت داده بود. وقتی گزارش درصد روی شیء جدا اجرا شد، گزینه ۶ دوباره همان جامی را نشان داد که کاربر با گزینه ۴ ساخته بود. به نظرم این بخش بهترین نمونه پروژه است که نشان می‌دهد تقسیم درست کلاس‌ها فقط ظاهر کد را مرتب نمی‌کند و مستقیماً جلوی خطای منطقی را می‌گیرد.

منابع مورد استفاده

هفت فایل آموزشی درس برنامه‌سازی پیشرفته و مثال‌های کدنویسی داخل آن‌ها.

فایل ۱۷ صفحه‌ای شرح پروژه ۵ شبیه‌ساز جام جهانی و قوانین خواسته شده.

نسخه چندماژولی برنامه، آزمون‌های پروژه و فایل داده ۳۲ تیم.